

Baseline loss rate with no UAVs

How often the home base is destroyed on a 100×100 map when nothing is flown against the fire. The reference point every later comparison is measured against.

Experiment	20260810_baseline
Date	10 August 2026
Commit	34276b2db4bf2eab0d506ba3830bd57af6cc67ca
Simulator	Wildfire-UAVSim, mesa 1.x
Main arm	5 000 runs, seeds 1 000 000 – 1 004 999
Control arm	1 000 runs, seeds 2 000 000 – 2 000 999
Interval	Wilson score, 95%

Summary

With the twelve parameters below fixed and `NUM_AGENTS = 0`, the wildfire destroys the home base in **every one of 5 000 independently seeded runs**. The loss rate is **100.0%**, 95% Wilson interval **99.92 – 100.00**.

The result is not marginal. The fire front advances at 1.94 cells per step, the base sits 73 cells from the farthest corner of the grid, and the slowest run of the 5 000 was lost at step 54 — 46 steps inside the 100 step budget. Nothing about the outcome depends on where the fire started: the loss rate is 100% in every band of ignition distance, including the outermost.

A saturated measurement cannot distinguish "always lost" from "the outcome field is stuck", so a control arm re-ran the same configuration truncated to 20 steps, on seeds the main arm never touched. It lost 33.1% of runs (95% Wilson 30.3 – 36.1) against 31.3% predicted from the main arm's step-of-loss distribution. The measurement discriminates; the scenario is simply unwinnable without UAVs.

Contents

1. Objective and result
2. The simulation model
3. Parameters
4. Method
5. Statistical treatment
6. Results
7. Validity
8. Reproducing

1 Objective and result

Every claim made about a UAV policy in this project is a claim about a difference: the base survived *more often than it otherwise would have*. That comparison needs a fixed, measured, no-treatment arm, and this experiment is it. The question is narrow — on a 100×100 map, with the base enabled and no UAVs in the air, how often is the run lost?

Loss rate = 100.00% (5 000 / 5 000 runs), 95% Wilson interval **99.92% – 100.00%**.

A run is *lost* when the home base has burned for `BHP = 4` cumulative steps. A run is *won* by reaching step `BATCH_SIZE = 100` with the base still standing. There is no third outcome once the firefighting extension is on.

Read the interval as the one-sided statement it effectively is. The point estimate sits on the boundary of the parameter space, so the interval has no room above it; the informative half is the lower bound. With 5 000 runs and no survivors, the data are consistent with a true loss rate as low as 99.92% and no lower. In practical terms: if this configuration can be survived at all, it is survived less than once in 1 300 attempts.

WHAT THIS BUYS THE COMPARISONS THAT FOLLOW

A no-treatment arm at exactly 100% is the most useful shape this measurement could have taken. It means every subsequent win under this configuration is attributable to the UAVs, with no baseline survival to subtract and no confounding by lucky ignition positions — there are no lucky ignition positions. A policy's win rate *is* its effect size.

It also means this configuration cannot rank policies at the bottom of the range: it has no headroom below. That is a property of the baseline, not a defect in it, and §7 says what to do about it.

2 The simulation model

Only the parts of the model that are live in this experiment are described here. The UAVs, their policies, the fuel and positioning-error extensions and the managing system are all inert with `NUM_AGENTS = 0`, and the wind and smoke were switched off. What is left is a fire, a forest and a base.

2.1 THE FOREST

The grid is 100×100 cells. Each cell independently receives a `Fire` agent — a tree — with probability `DENSITY_PROB`. At the value used here, `1.0`, every cell holds one: the map is fully wooded and the fire never encounters a firebreak. Each tree carries fuel drawn uniformly from `[FUEL_BOTTOM_LIMIT, FUEL_UPPER_LIMIT] = [5, 10]` and loses `BURNING_RATE = 1` unit per fire update, so a burning cell stays alight for 5 to 10 updates before burning out and becoming inert.

2.2 SPREAD

The fire updates every `FIRE_SPREAD_SPEED = 1` steps, which is every step: the fastest spread the model offers. An unburnt cell s catches fire with probability

$$P(s) = 1 - \prod_{s' \text{ burning}} (1 - w(s, s'))$$
$$w(s, s') = d(s, s')^{-2} \quad \text{for } d \leq 3, \text{ else } 0 \quad // \text{ Moore neighbourhood, radius } 3$$

The four orthogonal neighbours are at $d = 1$, giving $w = 1$: a cell with a burning orthogonal neighbour ignites with certainty. Diagonals and the radius-2 and radius-3 ring contribute fractionally. The whole grid is evaluated in one convolution rather than a per-cell loop (`sim/fire_spread.py`), which is why 5 000 runs of a fully wooded 100×100 map cost 897 seconds of CPU rather than a day of it.

With `ACTIVATE_WIND = False` the kernel is isotropic and consumes no randomness at all, so spread is symmetric in every direction. The measured consequence, from §6.3, is a front that advances at **1.94 cells per step** — faster than the orthogonal guarantee of 1, because the radius-3 tail ignites cells ahead of the front.

2.3 IGNITION

`FIRE_START_POSITION = "random"` draws the ignition cell uniformly over the grid, excluding the base footprint, using the seeded generator. `FIRE_START_STEP = 0` lights it before the first step is taken, so the fire is already burning in the state the model is built in. Fixing the start step removes one source of variance that has nothing to do with the treatment under test; the ignition *position* is deliberately left random, because that is the scenario variation the baseline is supposed to average over.

2.4 THE BASE AND THE LOSS CONDITION

`ACTIVATE_FIREFIGHTING = True` puts a home base on the map. With `BASE_POSITION = None` it is anchored a quarter of the way into the grid, at cell `(25, 25)`, and covers a `BASE_SIZE = (2, 2)` footprint: cells `(25,25) (25,26) (26,25) (26,26)`. The footprint burns like any other part of the map.

The base accumulates one point of damage for every step on which it is burning, and the run is lost the moment that total reaches `BHP = 4`. The damage is cumulative rather than consecutive, which matters because a burning cell goes out for a step when none of its neighbours are alight yet. All 5 000 losses recorded exactly 4 burning steps, confirming the threshold rather than some other stopping condition ended each run.

The rest of the firefighting extension — water, refills, extinguishing, re-ignition, out buildings — requires a UAV to act. With none in the air, none of it fires. `buildings_total` is 0 in every run.

2.5 WHAT WAS SWITCHED OFF, AND WHY

INERT OR DISABLED SUBSYSTEMS

SUBSYSTEM	STATE	REASON
UAVs	0 agents	the treatment being withheld
policy	inert	nothing to allocate a policy to
managing system	none	no team to adapt
wind	off	asked for; also makes spread isotropic and consumes no randomness
smoke	off	asked for; smoke only occludes observation, and nothing observes
fuel extension	inert	a UAV-side extension
positioning error	inert	a UAV-side extension

The UAV-side extensions were left at their `config.py` defaults rather than explicitly disabled, because with no UAVs their settings are never read. `config-used.json` records the value of all 90 settings as they stood, so a later reader can check that assumption instead of taking it on trust.

3 Parameters

Twelve settings were fixed on the command line. Everything else took its `config.py` default at the recorded commit, which is itself a calibrated set — see the 20260810 calibration report.

FIXED PARAMETERS – THE EXPERIMENTAL SPECIFICATION

SPECIFIED AS	SETTING	VALUE	DEFAULT WAS
Grid size 100×100	<code>WIDTH</code> , <code>HEIGHT</code>	100, 100	100, 100
Base enabled	<code>ACTIVATE_FIREFIGHTING</code>	True	True
Base HP 4	<code>BHP</code>	4	4
Fire start position random	<code>FIRE_START_POSITION</code>	"random"	"random"
Fire start step 0, fixed	<code>FIRE_START_STEP</code>	0	(10, 20)
Batch size 100	<code>BATCH_SIZE</code>	100	100
Burning rate 1	<code>BURNING_RATE</code>	1	1
Fire spread speed 1	<code>FIRE_SPREAD_SPEED</code>	1	1
Vegetation density 1	<code>DENSITY_PROB</code>	1.0	0.9
Wind none	<code>ACTIVATE_WIND</code>	False	True
Smoke none	<code>ACTIVATE_SMOKE</code>	False	True
No UAVs	<code>NUM_AGENTS</code>	0	4

Five of the twelve depart from the calibrated defaults, marked above. Three of them — density, wind and ignition step — make the scenario *harder* than the calibrated default in ways worth naming, because the calibration report's headline figures were measured under different values and do not transfer:

- `DENSITY_PROB` 0.9 → 1.0 removes every gap in the vegetation. At 0.9 roughly a tenth of cells cannot burn, which thins the front and occasionally stalls it; at 1.0 nothing stops it.
- `FIRE_START_STEP` (10, 20) → 0 gives the fire 10 to 20 more steps of spreading inside the same 100 step budget. Under the default it was also a source of run-to-run variance unrelated to any treatment.
- `ACTIVATE_WIND` True → False is not obviously a hardening, and in the calibration experiment it was the reverse: wind concentrates the fire into a ray that frequently misses a 2×2 base, so *stronger wind made the scenario easier*. Isotropic spread reaches the base from any ignition point.

`ACTIVATE_SMOKE` and `NUM_AGENTS` have no effect on the fire itself. Smoke in this model occludes observation and nothing else; with no observers it is a no-op either way, and it was switched off so that the specification and the run agree.

3.1 THE SETTINGS THAT WERE NOT SPECIFIED

Of the parameters left at default, the ones the result is actually sensitive to are the fuel limits (`FUEL_BOTTOM_LIMIT` = 5, `FUEL_UPPER_LIMIT` = 10), which set how long a cell burns, the spread radius of 3 baked into `sim/fire_spread.py`, and `BASE_POSITION` = None, which is what puts the

base at (25, 25) rather than in the middle. A base at the centre of the grid would be nearer the average ignition point, not further, so that choice is conservative with respect to the headline.

4 Method

4.1 INDEPENDENCE

Runs are executed by `headless.py` across 8 worker processes. Run i of the batch is seeded `base_seed + i`, and `seed_simulation()` replaces `config.SYSTEM_RANDOM` — the single generator the whole simulation draws from — with a fresh `random.Random(seed)` before the model is built. Nothing is carried from one run into the next: no state, no generator, no worker-local caching of the grid. The runs are independent draws from the same distribution, which is the condition the binomial model in §5 requires.

Two properties of the arrangement are worth stating explicitly, because both have been wrong in this repository before. First, the seed each run used is recorded in its own result record, and the analysis script refuses to summarise a batch whose seeds are not distinct — a parallel batch that silently reused a seed would inflate n without adding information. Second, seeding is process-global, so the process executor is the only one that can run workers concurrently; the thread executor is limited to one worker for exactly this reason.

Consecutive integer seeds feed a Mersenne Twister through `random.Random(seed)`, whose initialisation scrambles them; adjacent seeds do not produce correlated streams. The evidence in this batch is direct: ignition positions are uniform over the grid, and the outcome is unanimous, so no seed-block artefact could be hiding in it.

4.2 SEED BLOCKS

ARM	N	SEEDS	PURPOSE
main	5 000	1 000 000 – 1 004 999	the reported loss rate
control	1 000	2 000 000 – 2 000 999	validity check, §6.4

Both blocks are disjoint from the blocks used by the 20260810 calibration experiment (1 000 for fitting, 500 000 for validation), so this baseline shares no run with the experiment that chose the defaults it is built on.

4.3 CHOICE OF N

5 000 was fixed before the batch was run, from a pilot of 8 runs which lost all 8 — enough to suggest the rate was near the boundary, not enough to estimate anything. Near $p = 1$ the useful quantity is the lower confidence bound, which for a unanimous sample is $n / (n + z^2)$ and depends on nothing but n :

WILSON LOWER BOUND FOR A UNANIMOUS SAMPLE

N	LOWER BOUND	READS AS
100	96.3%	weak; a 4% survival rate would not be excluded
1 000	99.62%	adequate
5 000	99.92%	survival excluded above 1 run in 1 300
20 000	99.98%	four times the compute for one decimal place

5 000 also keeps the interval useful in the case that did not happen. Had the true rate been, say, 97%, 5 000 runs would have given a half-width of about ± 0.5 points — fine enough to compare against any policy arm run at the usual $n = 1\,000$. The batch cost 897 seconds of CPU across 8 workers, about two and a half minutes of wall time.

4.4 ANALYSIS

`analyse.py` reads the raw JSON, refuses the batch if any run errored, if any run reports an outcome other than WON or LOST, or if two runs share a seed, and writes `results.csv` in long format — one row per run, with the ignition cell and its distance to the base joined on. The Wilson function is imported from `tools/sweep.py` rather than reimplemented, so this experiment and the repository's sweep tool cannot disagree about what a 95% interval is.

5 Statistical treatment

5.1 THE MODEL

Each run is a Bernoulli trial with unknown loss probability p , and the runs are independent by construction (§4.1), so the number of losses X in n runs is $\text{Binomial}(n, p)$. The estimator is the sample proportion $\hat{p} = X / n$.

5.2 WHY WILSON

The Wald interval, $\hat{p} \pm z\sqrt{\hat{p}(1-\hat{p})/n}$, collapses to a point at $\hat{p} = 1$: the standard error is exactly zero, and the interval it reports is [100%, 100%]. That is not a confidence statement, it is a division by the very thing being estimated. The Wilson score interval inverts the score test instead of assuming normality at the estimate, keeps its bounds inside $[0, 1]$, and returns a finite, correctly-shaped interval for a unanimous sample. It is the interval the repository already uses for its win rates, for the mirror-image reason: win rates near 0.1 with a few hundred trials are where Wald misbehaves in the other direction.

5.3 THE INTERVAL

$$\text{centre} = \frac{\hat{p} + z^2/2n}{1 + z^2/n}, \quad \text{half} = \frac{z}{1 + z^2/n} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n} + \frac{z^2}{4n^2}}$$

CI = centre $\pm$ half, clamped to $[0, 1]$ $z = 1.96$ for 95%

Worked through for the main arm, with $X = n = 5000$ and $z = 1.96$:

$$\begin{aligned}\hat{p} &= 5000 / 5000 &&= 1.0 \\ z^2/n &= 3.8416 / 5000 &&= 0.00076832 \\ \text{centre} &= (1 + 0.00038416) / 1.00076832 &&= 0.99961614 \\ \text{half} &= (1.96 / 1.00076832) \cdot \sqrt{(0 + 3.8416/10^8)} &&= 1.95849 \cdot 0.00019600 = \\ &&&0.00038384 \\ \text{CI} &= [0.99923, 0.99999998] \rightarrow \text{clamped} &&[99.92\%, 100.00\%]\end{aligned}$$

Both terms under the root are worth noticing. The first, $\hat{p}(1-\hat{p})/n$, is zero — it is the term Wald has and nothing else. The whole width of this interval comes from the second, $z^2/4n^2$, the continuity-like term Wilson carries and Wald does not. That is precisely why the interval is reportable at all.

5.4 WHAT IS NOT CLAIMED

- The interval covers sampling error in p under this configuration. It says nothing about whether the configuration is the right one, which is a modelling question, not a statistical one.

- The per-band intervals in §6.2 are four intervals on the same data; no multiplicity correction is applied, and none is needed, because they are descriptive rather than a family of tests.
- The control arm in §6.4 is a validity check, not a hypothesis test. Its interval is reported so the agreement with the prediction can be judged, not so a p -value can be computed from it.

6 Results

6.1 HEADLINE

MAIN ARM — 5 000 RUNS, NO UAVS

ARM	N	LOST	WON	LOSS RATE	95% WILSON
no UAVs	5 000	5 000	0	100.00%	99.92 — 100.00

6.2 LOSS RATE AGAINST IGNITION DISTANCE

Distance is Chebyshev — the number of spread events separating the ignition cell from the nearest base cell — because the fire spreads through a Moore neighbourhood. The largest distance available on this grid, from the far corner, is 73.

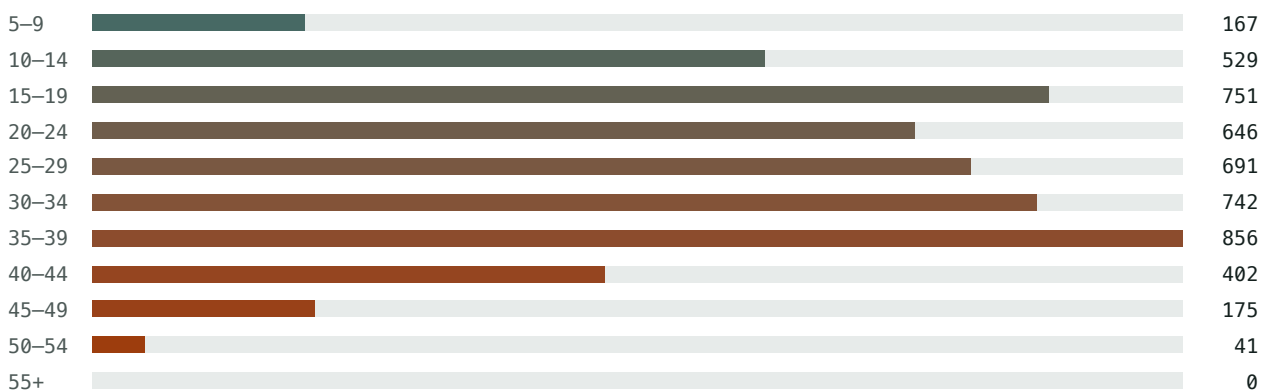
BY DISTANCE FROM IGNITION TO THE BASE FOOTPRINT

DISTANCE	N	LOST	LOSS RATE	95% WILSON	MEAN STEP OF LOSS
[0, 20)	811	811	100.00%	99.53 — 100.00	12.0
[20, 40)	1 442	1 442	100.00%	99.73 — 100.00	20.4
[40, 60)	1 439	1 439	100.00%	99.73 — 100.00	31.1
[60, 73]	1 308	1 308	100.00%	99.71 — 100.00	39.7

The outermost band is the one that could have failed and did not. Ignitions 60 or more cells from the base — a quarter of the grid away, in the far corners — still destroy it in every one of 1 308 runs, and still with 46 steps to spare. Distance changes *when* the base falls, not *whether* it does.

6.3 STEP OF LOSS, AND THE SPEED OF THE FRONT

DISTRIBUTION OF THE STEP THE RUN WAS LOST ON (N = 5 000)



Bar length is relative to the largest bin, 856 runs in the 35–39 band. No run survived past step 54.

STEP OF LOSS	MIN	MEDIAN	MEAN	95TH PCT	MAX	BUDGET
steps	5	27	27.2	44	54	100

Regressing the step of loss on the ignition distance over all 5 000 runs gives

$$\text{step_of_loss} = 5.34 + 0.5161 \cdot \text{distance} \quad \text{Pearson } r = 0.964$$

$$1 / 0.5161 = 1.94 \text{ cells per step} \quad \text{the speed of the fire front}$$

The two coefficients are both interpretable, which is the reason to report them. The slope inverts to a front speed of 1.94 cells per step — faster than the orthogonal certainty of one cell per step, because the radius-3 tail of the kernel lights cells ahead of the front. The intercept of 5.34 steps is the fixed cost at the base once the fire arrives: $\text{BHP} = 4$ burning steps plus roughly one step of arrival and re-ignition. An r of 0.964 says the ignition position determines almost the whole outcome trajectory, with the stochastic part of the spread contributing only the remainder.

The 100 step budget is not what decides this arm. The slowest of 5 000 runs was lost at step 54, against a budget of 100, and the ignition distances sampled reach the maximum the grid allows. The loss rate would still be 100% on these seeds at any `BATCH_SIZE` from 54 upwards, and the margin is wide enough that a modest extra allowance for stochastic variation changes nothing. The result therefore does not turn on the run-length calibration, which is the one parameter the calibration report calls the strongest difficulty setting there is.

6.4 CONTROL ARM: DOES THE MEASUREMENT DISCRIMINATE?

A rate of exactly 100% with zero variance is the same reading a broken outcome field would give. The control arm distinguishes them. It repeats the configuration unchanged except for `BATCH_SIZE = 20`, on 1 000 seeds from a block the main arm never touched. Twenty steps is inside the body of the step-of-loss distribution, so a substantial share of runs should end with the base still standing — and the share that should not is predictable from the main arm without running anything: it is the fraction of main-arm runs lost at or before step 20.

CONTROL ARM — SAME CONFIGURATION TRUNCATED TO 20 STEPS

SOURCE	N	LOST	LOSS RATE	95% WILSON
predicted from main arm	5 000	1 567	31.34%	—
observed, seeds 2 000 000+	1 000	331	33.10%	30.25 — 36.08

The prediction falls inside the observed interval. Three things follow. The outcome field records WON when a run is won, so the unanimity in §6.1 is a property of the scenario and not of the instrument. The two seed blocks agree, so neither carries a block-specific artefact. And the step-of-loss distribution in §6.3 is calibrated well enough to predict an independent experiment, which is what licenses the run-length claim in the box above.

7 Validity

7.1 THE BASELINE HAS NO HEADROOM BELOW

This is the one limitation that will bite. A no-treatment arm at 100% cannot rank interventions that all fail: a policy that saves 0 runs in 1 000 and a policy that saves 1 are separated by this baseline hardly at all. It is an excellent reference for *attribution* — every win is the UAVs' — and a poor one for *discrimination* at the weak end of the range.

If a later experiment needs discrimination there, the lever is not the baseline but the difficulty. The calibration report ranks the dials; the three fixed here that differ from the calibrated defaults — density 1.0, ignition at step 0, wind off — are each worth several points of win rate and are the first places to look. Re-measuring this baseline is a prerequisite for any such change, since none of the numbers on this page transfer.

7.2 SCOPE

- **One configuration.** Every figure is conditional on the parameter set in §3 at commit `34276b2`. The calibration report's finding that a *larger* grid and a *stronger* wind both make the scenario easier is the standing warning against extrapolating any of these dials by intuition.
- **Ignition position is the only randomised scenario factor.** Fixing the start step at 0 was requested and reduces variance, but it also means this baseline says nothing about a fire that starts mid-run, which is the default configuration's behaviour.
- **The lower bound is a sampling bound.** 99.92% bounds sampling error at $n = 5\,000$. It does not bound the probability that the simulator itself has a defect that makes the base unsaveable, which no amount of sampling can address. §6.4 addresses the nearest version of that concern that is testable from outside.

7.3 THINGS THAT WERE CHECKED

- All 5 000 runs completed without error; `analyse.py` refuses the batch otherwise.
- All 5 000 seeds are distinct, so $n = 5\,000$ independent draws is the true sample size.
- All 5 000 losses recorded exactly `base_burning_steps = 4`, matching `BHP`: the runs ended on the loss condition and not on some other stopping rule.
- Ignition distances span 1 to 73, the full range the grid allows, with no gap in the distribution.
- The control arm registers wins, on an independent seed block, at the predicted rate.

8 Reproducing

From the repository root, at commit `34276b2db4bf2eab0d506ba3830bd57af6cc67ca`:

```
# main arm -- 5 000 runs, no UAVs, about 2.5 minutes of wall time on 8 workers
python3 headless.py \
  --runs 5000 --workers 8 --seed 1000000 --policy random --managing none \
  --log-every 0 --log-level ERROR \
  --set WIDTH=100 --set HEIGHT=100 --set BATCH_SIZE=100 --set NUM_AGENTS=0 \
  --set ACTIVATE_FIREFIGHTING=True --set BHP=4 \
  --set 'FIRE_START_POSITION=random' --set FIRE_START_STEP=0 \
  --set BURNING_RATE=1 --set FIRE_SPREAD_SPEED=1 --set DENSITY_PROB=1.0 \
  --set ACTIVATE_WIND=False --set ACTIVATE_SMOKE=False \
  --output experiments/20260810_baseline/runs/no-uav.json

# control arm -- same, truncated to 20 steps, on a disjoint seed block
python3 headless.py \
  --runs 1000 --workers 8 --seed 2000000 --policy random --managing none \
  --log-every 0 --log-level ERROR \
  --set WIDTH=100 --set HEIGHT=100 --set BATCH_SIZE=20 --set NUM_AGENTS=0 \
  ... (the remaining --set flags unchanged) \
  --output experiments/20260810_baseline/runs/control-20-steps.json

# analysis: results.csv and summary.json
python3 experiments/20260810_baseline/analyse.py

# the full configuration as it was applied, for a later diff
python3 experiments/20260810_baseline/dump_config.py
```

`--policy random` is recorded for completeness only; with `NUM_AGENTS = 0` no policy is ever consulted. Set the run length through `BATCH_SIZE`: the model stops itself at its own `BATCH_SIZE`, and `--steps` is an alias for the same setting.

FILES

PATH	CONTENTS
runs/no-uav.json	raw headless output, 5 000 records
runs/control-20-steps.json	raw headless output, 1 000 records
results.csv	long format, one row per main-arm run, distance joined on
summary.json	every number quoted in §6
config-used.json	all 90 settings as applied, plus the commit
analyse.py	produces results.csv and summary.json
dump_config.py	produces config-used.json
baseline-report.html / .pdf	this document, and its source